B I T Á C O R A

Saga orquestada y circuit breaker

Mexi Banco · v06 · Transferencia entre microservicios

Fecha de las pruebas: 17 de octubre de 2026

Nombre: Guerrero López Uriel Iván

Contenido

- 1. El problema (Parte 0)
- 2. La saga y su diagrama de estados
- 3. El código clave
- 4. La compensación funcionando
- 5. Cuando la compensación también falla
- 6. El circuit breaker
- 7. Lo que la saga no da: aislamiento
- 8. Limitaciones
- Anexo: archivos entregados

1. El problema: dinero “en el aire”

Después de partir el monolito, una transferencia ya no es una sola transacción: el cargo y el abono son dos llamadas HTTP a otro servicio, cada una confirmada por separado. Para reproducirlo, levanté transferencia con una pausa de 20 segundos entre el cargo y el abono, lancé una transferencia de 100 pesos y apagué cuenta durante la pausa.

Cuenta	Saldo antes	Saldo después
Ana (origen) 002180009223841501	1000.00	900.00
Beto (destino) 002180009223841502	500.00	500.00 (no recibió nada)

```
$ curl -i http://localhost:8080/api/cuentas/002180009223841501
{"id":4,"clabe":"002180009223841501","titular":"Ana","saldo":1000.00}
$ curl -i http://localhost:8080/api/cuentas/002180009223841502
{"id":5,"clabe":"002180009223841502","titular":"Beto","saldo":500.00}


$ PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia
$ docker compose exec transferencia printenv PAUSA_ENTRE_CARGO_Y_ABONO_MS
20000


$ curl -i -X POST http://localhost:8080/api/transferencias \
    -H "Content-Type: application/json" \
    -d '{"claveOrigen":"002180009223841501","claveDestino":"002180009223841502","monto":100}'
HTTP/1.1 504 Gateway Timeout
{"status":504,"error":"Gateway Timeout","path":"/api/transferencias"}


$ curl -s http://localhost:8080/api/cuentas/002180009223841501   # tras reencender cuenta
{"id":4,"clabe":"002180009223841501","titular":"Ana","saldo":900.00}
```

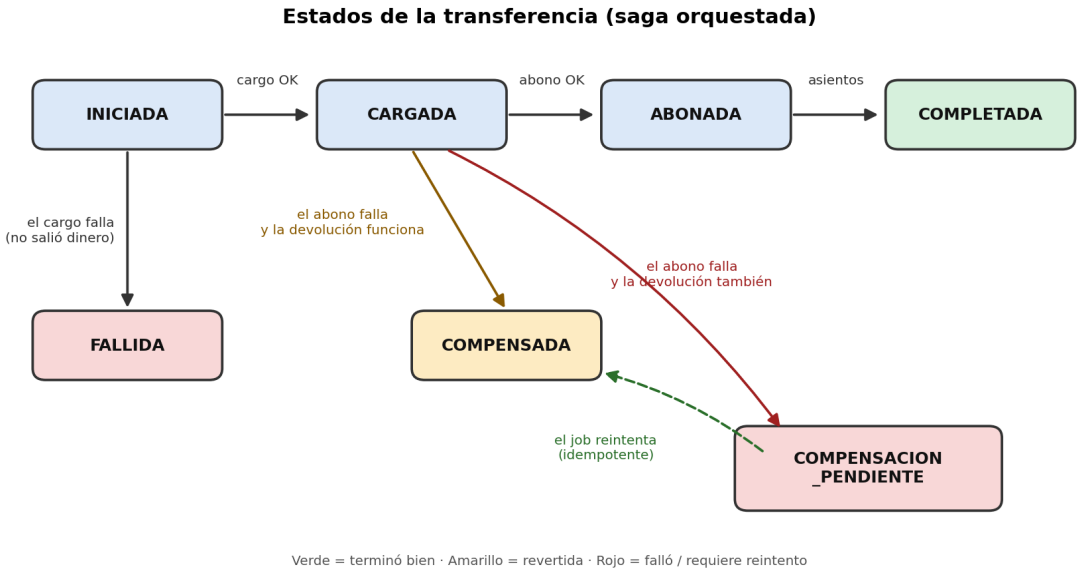
Captura 1. *Saldos antes y después de la Parte 0: salieron 100 pesos de Ana y no llegaron a Beto.*

Por qué pasó: el cargo ya estaba confirmado en la base de cuenta cuando esta se apagó, y como cargo y abono no comparten una transacción, nadie deshizo el cargo.

2. La saga: pasos, compensaciones y estados

TransferenciaService es el orquestador: decide qué paso sigue y qué hacer si uno falla. Cada transferencia tiene una fila con su estado, que se confirma en cada paso (el método no es @Transactional a propósito). Así, si el servicio se cae a la mitad, la base dice en qué paso se quedó.

Paso	Acción	Si falla
1. Cargo	Restar el monto al origen (Idempotency-Key transferencia-{id}-cargo)	FALLIDA: no salió dinero, no hay nada que revertir
2. Abono	Sumar el monto al destino (clave transferencia-{id}-abono)	Se compensa (paso 3)
3. Compensación	Abonar el monto de vuelta al origen y registrar un asiento COMPENSACION (clave transferencia-{id}-compensacion)	COMPENSACION_PENDIENTE: lo termina el job



Captura 2. *Diagrama de estados de la transferencia.*

El cliente no recibe un 500 sin más: obtiene un cuerpo con el id, el estado y un mensaje que explica lo ocurrido.

Estado final	HTTP	Significado
COMPLETADA	201	Cargo y abono hechos
COMPENSADA	409	No se hizo; el dinero volvió al origen
COMPENSACION_PENDIENTE	503	No se hizo; la devolución se reintenta sola

3. El código clave

3.1 La entidad guarda el estado

Transferencia ganó un estado, un detalle en texto, el número de intentos de compensación y la hora de la última actualización. Esta última sirve para que el job solo reintente filas que llevan un rato sin cambios.

```
@Enumerated(EnumType.STRING)
private EstadoTransferencia estado = EstadoTransferencia.INICIADA;
private String detalle;
private int intentosCompensacion;
private Instant actualizada = Instant.now();

public void cambiarEstado(EstadoTransferencia nuevo, String detalle) {
    this.estado = nuevo;
    if (detalle != null) this.detalle = detalle;
    this.actualizada = Instant.now();
}
```

3.2 El orquestador

Cada paso se confirma con transferencias.save(t) antes de pasar al siguiente. Se atrapan tres tipos de falla de infraestructura: ErrorRemotoException (cuenta respondió con error), ResourceAccessException (cuenta no contestó) e IllegalStateException (Eureka no tiene instancias de cuenta). Esta última apareció de verdad en las pruebas; sin ella la transferencia se habría quedado en CARGADA.

```
Transferencia t = transferencias.save(new Transferencia(origen, destino, monto)); // INICIADA
String base = "transferencia-" + t.getId();

origen = cuentas.cargar(claveOrigen, monto, base + "-cargo");
t.cambiarEstado(EstadoTransferencia.CARGADA, null);
t = transferencias.save(t);

try {
    destino = cuentas.abonar(claveDestino, monto, base + "-abono");
} catch (ErrorRemotoException | ResourceAccessException | IllegalStateException e) {
    return compensar(t, "Falló el abono a " + claveDestino + ": " + e.getMessage());
}
t.cambiarEstado(EstadoTransferencia.ABONADA, null);
t = transferencias.save(t);
t.cambiarEstado(EstadoTransferencia.COMPLETADA, null);
return transferencias.save(t);
```

3.3 La compensación

Devuelve el monto al origen con una clave de idempotencia fija por transferencia, registra el asiento COMPENSACION en movimiento y marca la transferencia. Si la devolución también falla, deja el estado COMPENSACION_PENDIENTE.

```
public Transferencia compensar(Transferencia t, String causa) {
    t.registrarIntentoCompensacion();
    try {
        CuentaRemota origen = cuentas.abonar(t.getClaveOrigen(), t.getMonto(),
            "transferencia-" + t.getId() + "-compensacion");
```

```
        asiento(origen.clabe(), TipoMovimiento.COMPENSACION, t.getMonto(), origen.saldo(),
            "reversa de transferencia " + t.getId());
        t.cambiarEstado(EstadoTransferencia.COMPENSADA, causa + ". Se devolvió el monto al
origen.");
    } catch (ErrorRemotoException | ResourceAccessException | IllegalStateException e) {
        t.cambiarEstado(EstadoTransferencia.COMPENSACION_PENDIENTE, causa + ". La devolución
también falló...");
    }
    return transferencias.save(t);
}
```

3.4 Idempotencia en cuenta

Mandar la clave no sirve de nada si cuenta no la respeta. cuenta guarda cada clave aplicada en la tabla operacion_aplicada (clave como llave primaria) en la misma transacción que actualiza el saldo. Si la clave ya existe, no mueve dinero y responde igual que la primera vez.

```
@Transactional
public Cuenta abonar(String clabe, BigDecimal monto, String clave) {
    if (clave == null) return abonar(clabe, monto);
    if (operaciones.existsById(clave)) return consultar(clabe);   // ya aplicado
    Cuenta cuenta = abonar(clabe, monto);
    operaciones.saveAndFlush(new OperacionAplicada(clave));       // misma transacción
    return cuenta;
}
```

3.5 La falla provocada sin apagar nada

La variable de entorno **FALLAR_ABONO_A_CLABE**: si la CLABE destino coincide, el abono falla con una falla simulada aunque cuenta esté arriba. Se pasa desde docker-compose y dentro del contenedor Spring la lee como mexibanco.demo.fallar-abono-a-clabe.

```
if (!fallarAbonoAClabe.isBlank() && fallarAbonoAClabe.equals(claveDestino)) {
    throw new ErrorRemotoException("cuenta", 503, "Falla simulada del abono hacia " +
claveDestino);
}


$ FALLAR_ABONO_A_CLABE=002180009223841502 docker compose up -d transferencia
```

3.6 cuenta y movimiento: los endpoints leen la clave y hay un tipo de asiento nuevo

El controller de cuenta recibe la clave en el header Idempotency-Key (opcional, para que spei siga funcionando igual). En movimiento solo agregué el tipo COMPENSACION al enum, que también copié en el cliente de transferencia.

```
@PostMapping("/{clabe}/abonos")
public Cuenta abonar(@PathVariable String clabe, @Valid @RequestBody Monto abono,
        @RequestHeader(value = "Idempotency-Key", required = false) String clave) {
    return cuentas.abonar(clabe, abono.monto(), clave);
}


public enum TipoMovimiento { DEPOSITO, TRANSFERENCIA_ENVIADA, TRANSFERENCIA_RECIBIDA,
SPEI_ENVIADO, COMPENSACION }
```

3.7 CuentaCliente protegido por el circuit breaker

Toda llamada a cuenta pasa por protegido(). Si el circuito no la permite, lanza un 503 sin tocar la red. Una falla de infraestructura o un 5xx suma al contador; un 404 o 422 cuenta como éxito porque cuenta respondió.

```
private <T> T protegido(Supplier<T> llamada) {
    if (!circuito.permitir())
        throw new ErrorRemotoException("cuenta", 503, "Circuito abierto: ... no se intento la
llamada");
    try {
        T r = llamada.get(); circuito.exito(); return r;
    } catch (ErrorRemotoException e) {
        if (e.getEstado() >= 500) circuito.fallo(); else circuito.exito();
        throw e;
    } catch (ResourceAccessException | IllegalStateException e) {
        circuito.fallo(); throw e;
    }
}
```

3.8 El job de reintento y el docker-compose

El job se salta la vuelta si el circuito está abierto, para no gastar intentos sin haber llamado a cuenta. En el compose solo agregué una línea al servicio transferencia: la variable que activa la falla simulada.

```
@Scheduled(fixedDelay = 15_000)
void reintentar() {
    if (!servicio.cuentaDisponible()) return;   // circuito abierto: se pospone
    Instant limite = Instant.now().minusSeconds(30);
    for (Transferencia t :
transferencias.findByEstadoAndActualizadaBefore(COMPENSACION_PENDIENTE, limite)) {
        if (t.getIntentosCompensacion() >= MAX_INTENTOS) { log.error("requiere intervención");
continue; }
        servicio.compensar(t, "Reintento programado");
    }
}

# docker-compose.yml, servicio transferencia > environment
MEXIBANCO_DEMO_FALLAR_ABONO_A_CLABE: ${FALLAR_ABONO_A_CLABE:-}
```

4. La compensación funcionando

Con cuenta arriba y la falla simulada hacia Beto, lancé una transferencia de 100 pesos. Ana estaba en 850.

```
$ docker compose exec transferencia printenv MEXIBANCO_DEMO_FALLAR_ABONO_A_CLABE
002180009223841502


$ curl -i -X POST http://localhost:8080/api/transferencias ... "monto":100
HTTP/1.1 409 Conflict
{"id":5,"estado":"COMPENSADA","mensaje":"Falló el abono a 002180009223841502: Falla
 simulada del abono hacia 002180009223841502. Se devolvió el monto al origen.",
 "claveOrigen":"002180009223841501","claveDestino":"002180009223841502","monto":100}
```

Captura 3. *Respuesta 409 con estado COMPENSADA y un mensaje que explica qué pasó.*

```
$ curl -s http://localhost:8080/api/cuentas/002180009223841501
{"id":4,"clabe":"002180009223841501","titular":"Ana","saldo":850.00}
$ curl -s http://localhost:8080/api/cuentas/002180009223841502
{"id":5,"clabe":"002180009223841502","titular":"Beto","saldo":550.00}
```

Captura 4. *Los saldos regresan: Ana en 850 y Beto en 550, igual que antes de la transferencia.*

```
$ curl -s "http://localhost:8080/api/movimientos?clabe=002180009223841501"
{"id":14,"tipo":"COMPENSACION","monto":100.00,"saldoResultante":850.00,
  "referencia":"reversa de transferencia 5"},
{"id":13,"tipo":"TRANSFERENCIA_ENVIADA","monto":-100.00,"saldoResultante":750.00,
  "referencia":"a 002180009223841502"},
...
```

Captura 5. *El ledger: el cargo de -100 (saldo 750) y, justo después, la devolución registrada como movimiento COMPENSACION de +100.*

```
$ docker compose exec db psql -U mexibanco -d transferencia -c "select id, estado,
intentos_compensacion, detalle ..."
 id |   estado    | intentos_compensacion |                   detalle
----+-------------+-----------------------+-----------------------------------------------
  3 | COMPLETADA  |                     0 |
  4 | COMPENSADA  |                     1 | Falló el abono a ...502: Falla simulada ... Se devolvió
el monto al origen.
  5 | COMPENSADA  |                     1 | Falló el abono a ...502: Falla simulada ... Se devolvió
el monto al origen.
```

Captura 6. *Estado final en la base: la transferencia 5 quedó en COMPENSADA con un intento. La fila 3 es el camino feliz (201, COMPLETADA).*

5. Cuando la compensación también falla

Repetí la Parte 0: pausa de 20 segundos, transferencia de 100 pesos y docker compose stop cuenta durante la pausa. El abono falla, la devolución también, y la transferencia no se queda en CARGADA: queda en COMPENSACION_PENDIENTE.

```
$ curl -i -X POST http://localhost:8080/api/transferencias ... "monto":100
HTTP/1.1 504 Gateway Timeout        <- el timeout del gateway venció durante la pausa;
                                       la saga siguió corriendo dentro de transferencia


$ docker compose exec db psql -U mexibanco -d transferencia -c "select id, estado,
intentos_compensacion, detalle ..."
  6 | COMPENSACION_PENDIENTE | 1 | Falló el abono a ...502: No instances available for cuenta.
                                   La devolución también falló (No instances available for cuenta);
                                   queda pendiente.
```

Captura 7. *Con cuenta apagada la transferencia 6 queda en COMPENSACION_PENDIENTE, no en CARGADA.*

Cómo la termino. Un job programado (RecuperadorDeCompensaciones) corre cada 15 segundos, busca transferencias en COMPENSACION_PENDIENTE sin cambios en los últimos 30 segundos y vuelve a llamar a compensar(). Tiene un tope de 10 intentos; al superarlo escribe un log de error para que lo atienda una persona. En producción añadiría espera creciente entre intentos y una alerta.

```
$ docker compose start cuenta
$ docker compose exec db psql ... -c "select ... where id = 6;"
  6 | COMPENSADA | 3 | Reintento programado. Se devolvió el monto al origen.


$ curl -s http://localhost:8080/api/cuentas/002180009223841501
{"id":4,"clabe":"002180009223841501","titular":"Ana","saldo":850.00}

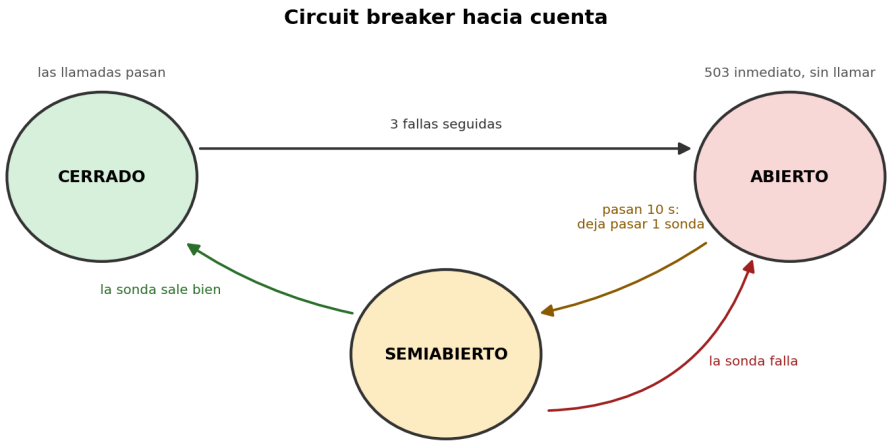

# ledger de Ana: un solo asiento COMPENSACION para la transferencia 6
{"id":16,"tipo":"COMPENSACION","monto":100.00,"saldoResultante":850.00,"referencia":"reversa de
transferencia 6"}
```

Captura 8. *Al volver cuenta, el job cerró la transferencia 6 tras 3 intentos. Ana quedó en 850, no en 950: la devolución se aplicó una sola vez.*

Por qué la compensación debe ser idempotente. Los reintentos son inevitables: el job corre varias veces, la respuesta de un intento puede perderse después de que cuenta ya aplicó el abono, y podría haber más de una instancia del job. Si cada intento devolviera el dinero otra vez, Ana terminaría con más de lo que tenía. Con la clave fija transferencia-{id}-compensacion, repetir la operación tiene el mismo efecto que hacerla una vez. La prueba lo demuestra: 3 intentos, una sola devolución. Reintentar sin idempotencia cobra (o devuelve) dos veces.

6. El circuit breaker

Lo implementé a mano (clase CircuitBreaker) y lo coloqué dentro de CuentaCliente, de modo que protege las tres llamadas de transferencia a cuenta: consultar, cargar y abonar, incluida la compensación. Con el circuito abierto se lanza un 503 sin llamar a cuenta, y los catch de la saga ya lo manejan.



Captura 9. *Los tres estados del circuito y qué provoca cada cambio.*

Estado	Qué hace	Cuándo cambia
CERRADO	Las llamadas pasan; cuenta fallas seguidas. Un éxito reinicia la cuenta.	A ABIERTO con 3 fallas seguidas
ABIERTO	Rechaza todo con 503 sin llamar a cuenta, durante 10 s.	A SEMIABIERTO al pasar los 10 s
SEMIABIERTO	Deja pasar una sola llamada de prueba (sonda).	Sonda buena: CERRADO. Sonda mala: ABIERTO

Qué cuenta como falla: caídas o timeouts (ResourceAccessException), falta de instancias en Eureka (IllegalStateException) y respuestas 5xx. Un 404 o 422 no cuenta: cuenta respondió bien, solo con un error de negocio.

```
public synchronized boolean permitir() {
    if (estado == CERRADO) return true;
    if (estado == ABIERTO) {
        if (reloj.getAsLong() - abiertoDesde >= esperaMs) { cambiar(SEMIABIERTO); sondaEnCurso
= true; return true; }
        return false;
    }
    if (sondaEnCurso) return false;   // SEMIABIERTO: una sonda a la vez
    sondaEnCurso = true; return true;
}
```

Evidencia 1: apagué cuenta y mandé seis transferencias

```
$ docker compose stop cuenta
$ for i in 1 2 3 4 5 6; do curl -s -o /dev/null -w "intento $i: HTTP %{http_code} en
%{time_total}s\n" ...; done
intento 1: HTTP 503 en 2.182729s
intento 2: HTTP 503 en 2.019370s
intento 3: HTTP 503 en 2.025569s
intento 4: HTTP 503 en 0.013516s
intento 5: HTTP 503 en 0.010689s
intento 6: HTTP 503 en 0.005111s

$ curl -s http://localhost:8080/api/transferencias/circuito
{"circuito":"cuenta","estado":"ABIERTO"}
```

Captura 10. *Los tres primeros intentos tardan unos 2 s (timeout de conexión); al abrirse el circuito responden en milisegundos.*

Intento	HTTP	Tiempo	Qué pasó
1	503	2.18 s	Llamó a cuenta y falló
2	503	2.02 s	Llamó a cuenta y falló
3	503	2.03 s	Tercera falla: el circuito se abre
4	503	0.014 s	Rechazado sin llamar a cuenta
5	503	0.011 s	Rechazado sin llamar a cuenta
6	503	0.005 s	Rechazado sin llamar a cuenta

Evidencia 2: encendí cuenta y el circuito se cerró

```
$ docker compose start cuenta
$ sleep 12 && curl -i -X POST http://localhost:8080/api/transferencias ... "monto":10
HTTP/1.1 201 Created
{"id":7,"estado":"COMPLETADA","mensaje":"Transferencia completada.",...,"monto":10}
$ curl -s http://localhost:8080/api/transferencias/circuito
{"circuito":"cuenta","estado":"CERRADO"}

$ docker compose logs transferencia | grep "Circuito 'cuenta'"
18:35:25.139 WARN CircuitBreaker : Circuito 'cuenta': CERRADO -> ABIERTO
18:35:45.797 WARN CircuitBreaker : Circuito 'cuenta': ABIERTO -> SEMIABIERTO
18:35:45.809 WARN CircuitBreaker : Circuito 'cuenta': SEMIABIERTO -> ABIERTO
18:36:23.640 WARN CircuitBreaker : Circuito 'cuenta': ABIERTO -> SEMIABIERTO
18:36:23.736 WARN CircuitBreaker : Circuito 'cuenta': SEMIABIERTO -> CERRADO
```

Captura 11. *Logs del circuito con los tres estados y los cuatro caminos posibles.*

Hora	Transición	Causa
18:35:25	CERRADO → ABIERTO	3 fallas seguidas con cuenta apagada
18:35:45	ABIERTO → SEMIABIERTO	Pasaron 10 s; la siguiente llamada fue la sonda
18:35:45	SEMIABIERTO → ABIERTO	La sonda falló: cuenta seguía apagada
18:36:23	ABIERTO → SEMIABIERTO	Nueva sonda, con cuenta ya encendida
18:36:23	SEMIABIERTO → CERRADO	La sonda salió bien (201, transferencia 7)

Interacción con la saga. Con el circuito abierto, un cargo falla con 503 sin mover dinero, y un abono pasa a compensar. Si la compensación también choca con el circuito, queda en COMPENSACION_PENDIENTE.

El job de reintento se salta la vuelta mientras el circuito esté abierto, para no gastar sus 10 intentos en rechazos que ni siquiera llegaron a cuenta.

Timeout explícito. ClientesHttp fija 2 s de conexión y 5 s de lectura; los 2 s de los primeros intentos lo demuestran. **Reintento con espera creciente:** no lo implementé; sería seguro porque cargo, abono y compensación llevan clave de idempotencia.

7. Lo que la saga no da: aislamiento

Una saga garantiza que la operación termina completa o compensada, pero no da aislamiento (la "I" de ACID). Cada paso se confirma al instante y es visible para cualquiera mientras la saga sigue en curso. Quien consulta el saldo de origen entre el cargo y la compensación ve un estado intermedio que nunca debió existir: dinero que ya salió de Ana y todavía no llegó a Beto, y que quizá se devuelva.

```
# ledger de Ana durante la transferencia 5 (con falla simulada)
18:17:42  TRANSFERENCIA_ENVIADA  -100.00  saldoResultante  750.00   <- cargo confirmado
          ... ventana en la que cualquiera que consulte ve 750 ...
18:17:43  COMPENSACION           +100.00  saldoResultante  850.00   <- la devolución llega
```

Captura 12. *Durante esa ventana, Ana aparece con 750 aunque la transferencia terminará revertida.*

Las consecuencias son concretas. Otra operación podría rechazarse por "saldo insuficiente" sobre dinero que en realidad volverá, o alguien podría tomar una decisión (mostrar el saldo en una app, aprobar un crédito) con una cifra que luego cambia. Con la pausa de 20 s de la Parte 0 esa ventana es de segundos; con cuenta caída y la compensación pendiente puede durar minutos. Incluso la lectura de Beto es engañosa: no ve nada hasta que el abono ocurre.

Es consistencia eventual: el sistema converge a un estado correcto, pero no es correcto en cada instante. Se mitiga con bloqueos semánticos (por ejemplo, marcar el dinero como "reservado" en lugar de restarlo, y mostrar el estado de la transferencia pendiente junto al saldo), pero eso es trabajo adicional que la saga por sí sola no hace.

Estado final de la tabla transferencia y saldos

id	Estado	Intentos	Prueba
3	COMPLETADA	0	Camino feliz
4	COMPENSADA	1	Falla simulada (sin asiento COMPENSACION)
5	COMPENSADA	1	Falla simulada (con asiento)
6	COMPENSADA	3	cuenta apagada; cerrada por el job
7	COMPLETADA	0	Sonda buena del circuito

Saldos finales: Ana 840.00 y Beto 560.00. Suman 1400, igual que al inicio: no se perdió ni se duplicó dinero.

Anexo: archivos modificados y entregados

La entrega incluye las carpetas completas de los tres servicios tocados (sin target/), el docker-compose.yml y esta bitácora.

Servicio	Archivo	Cambio
transferencia	EstadoTransferencia.java	Nuevo: los siete estados de la saga
transferencia	Transferencia.java	Estado, detalle, intentos y hora de actualización
transferencia	TransferenciaRepository.java	Búsqueda por estado y antigüedad
transferencia	TransferenciaService.java	Orquestador de la saga y compensación
transferencia	TransferenciaController.java	Respuesta con estado y mensaje (201/409/503) y endpoint /circuito
transferencia	RecuperadorDeCompensaciones.java	Nuevo: job de reintento
transferencia	clientes/CircuitBreaker.java	Nuevo: circuit breaker a mano con tres estados
transferencia	clientes/CuentaCliente.java	Clave de idempotencia y circuit breaker
transferencia	clientes/TipoMovimiento.java	Agrega COMPENSACION
transferencia	tests: TransferenciaServiceTest, ClientesHttpTest, CircuitBreakerTest	Ajustados y nuevos
cuenta	OperacionAplicada y su repositorio	Nuevos: registro de claves aplicadas
cuenta	CuentaService.java, CuentaController.java	Operaciones idempotentes y lectura del header
movimiento	TipoMovimiento.java	Agrega COMPENSACION
raíz	docker-compose.yml	Variable de la falla simulada en transferencia